

HIGH GARDEN COFFEE

Analítica de Consumo Doméstico de Café (1990-2020)

Reto técnico de ingeniería de Machine Learning — NTT DATA

Joe Louis Arroyo Lobo

NTT DATA

3 de septiembre de 2026

Tabla de Contenido

Introducción	3
1. Decisiones de preparación de datos	4
2. Análisis exploratorio y tendencias	6
3. Crecimiento y ranking de países (CAGR)	10
4. Segmentación de países productores (clustering)	12
5. Forecasting de consumo doméstico	14
6. Índice de presión de precios (SIMULADO)	17
7. Detección de anomalías y eventos relevantes	19
8. Conclusiones y recomendaciones de negocio	21
9. Bonus: chatbot con IA generativa	22

Introducción

High Garden Coffee es una exportadora internacional de café. El área de innovación busca aprovechar una base histórica de consumo doméstico de café (1990-2020, 55 países, distintos tipos de café) para identificar tendencias, información clave y una posible señal de precios futuros que den ventaja competitiva.

La idea central del análisis es la siguiente: para una exportadora, el consumo doméstico en los países productores es la otra cara del excedente exportable. A más consumo doméstico, menos café disponible para exportar (todo lo demás constante). Por ello, el análisis recorre: tendencias, crecimiento por país, segmentación de países productores, forecasting de consumo, un índice de presión de precios (simulado, con metodología explícita) y detección de eventos anómalos, cerrando con recomendaciones de negocio y una propuesta bono de chatbot con IA generativa.

Código 1. Configuración del entorno y librerías

```
import warnings
warnings.filterwarnings('ignore')

import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from statsmodels.tsa.holtwinters import ExponentialSmoothing

sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (10, 5)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Rutas relativas robustas: no dependen del directorio desde el que se
# invoque el script
BASE_DIR = Path.cwd()
DATA_PATH = BASE_DIR / 'coffee_db.parquet'
```

```
CHARTS_DIR = BASE_DIR / 'charts'
OUTPUTS_DIR = BASE_DIR / 'outputs'
CHARTS_DIR.mkdir(exist_ok=True)
OUTPUTS_DIR.mkdir(exist_ok=True)
```

Resultado en consola:

```
DATA_PATH existe: True
```

Código 2. Carga del dataset

```
df_wide = pd.read_parquet(DATA_PATH)
print('Shape:', df_wide.shape)
print('Nulos por columna:', df_wide.isnull().sum().sum())
df_wide.head()
```

Resultado en consola:

```
Shape: (55, 33)
Nulos por columna: 0
```

Tabla 1

Vista previa del conjunto de datos original (formato ancho)

Country	Coffee type	1990/91	1991/92	1992/93	1993/94
Angola	Robusta/Arabica	1,200,000	1,800,000	2,100,000	1,200,000
Bolivia (Plurinational State of)	Arabica	1,500,000	1,620,000	1,650,000	1,710,000
Brazil	Arabica/Robusta	492,000,000	510,000,000	534,000,000	546,000,000
Burundi	Arabica/Robusta	120,000	96,000	102,000	114,600
Ecuador	Arabica/Robusta	21,000,000	21,000,000	21,000,000	21,000,000

Nota. Se muestran las primeras columnas a modo ilustrativo. El conjunto completo tiene 33 columnas: País, Tipo de café y 30 temporadas (1990/91 a 2019/20), para 55 países.

1. Decisiones de preparación de datos

Tipo de café (Coffee type): el conjunto de datos trae cuatro valores: Arabica, Robusta, Arabica/Robusta y Robusta/Arabica. El orden de las etiquetas mixtas no está documentado de forma confiable en la fuente como indicador de variedad predominante, así que para el análisis principal (tendencias, clustering, forecasting por tipo) se agrupan Arabica/Robusta y

Robusta/Arabica en una sola categoría “Mixto”. Se conserva la columna original Coffee type sin modificar, por si en el futuro se confirma que el orden sí es significativo.

Unidades de consumo: el enunciado del reto indica que el consumo está expresado “en tazas”, pero la estructura de esta serie (55 países productores, 1990-2020, valores redondeados a miles) corresponde al formato histórico de consumo doméstico que publica la Organización Internacional del Café (OIC), habitualmente expresado en miles de sacos de 60 kg. No se aplica ninguna conversión sin confirmar la fuente exacta: los ejes y las tablas se etiquetan de forma neutral (“unidades reportadas por la fuente”) para no inducir un error de escala.

Formato: los datos vienen en formato ancho (una columna por temporada). Se transforman a formato largo (Country, Coffee type, Coffee_type_grouped, Season, Year_start, Consumption) para el análisis de series de tiempo.

Código 3. Agrupación de tipo de café y transformación a formato largo

```
SEASON_COLS = [c for c in df_wide.columns if '/' in c and c[:2].isdigit()]
print(f'{len(SEASON_COLS)} temporadas: {SEASON_COLS[0]} .. {SEASON_COLS[-1]}')

def coffee_type_group(t: str) -> str:
    t = t.strip()
    if t in ('Arabica/Robusta', 'Robusta/Arabica'):
        return 'Mixto'
    return t

df_wide['Coffee_type_grouped'] = df_wide['Coffee
type'].apply(coffee_type_group)
print(df_wide['Coffee_type_grouped'].value_counts())

df_long = df_wide.melt(
    id_vars=['Country', 'Coffee type', 'Coffee_type_grouped'],
    value_vars=SEASON_COLS,
    var_name='Season',
    value_name='Consumption',
)
df_long['Year_start'] = df_long['Season'].str.slice(0, 4).astype(int)
df_long = df_long.sort_values(['Country',
'Year_start']).reset_index(drop=True)
df_long.head()
```

Resultado en consola:

```

30 temporadas: 1990/91 .. 2019/20
Coffee_type_grouped
Arabica      22
Mixto        18
Robusta      15
Name: count, dtype: int64

```

Tabla 2

Vista previa del conjunto de datos transformado a formato largo

Country	Coffee type	Coffee_type_grouped	Season	Consumption	Year_start
Angola	Robusta/Arabica	Mixto	1990/91	1,200,000	1990
Angola	Robusta/Arabica	Mixto	1991/92	1,800,000	1991
Angola	Robusta/Arabica	Mixto	1992/93	2,100,000	1992
Angola	Robusta/Arabica	Mixto	1993/94	1,200,000	1993
Angola	Robusta/Arabica	Mixto	1994/95	1,500,000	1994

2. Análisis exploratorio y tendencias

Código 4. Tendencia global de consumo

```

global_by_year =
df_long.groupby('Year_start')['Consumption'].sum().reset_index()

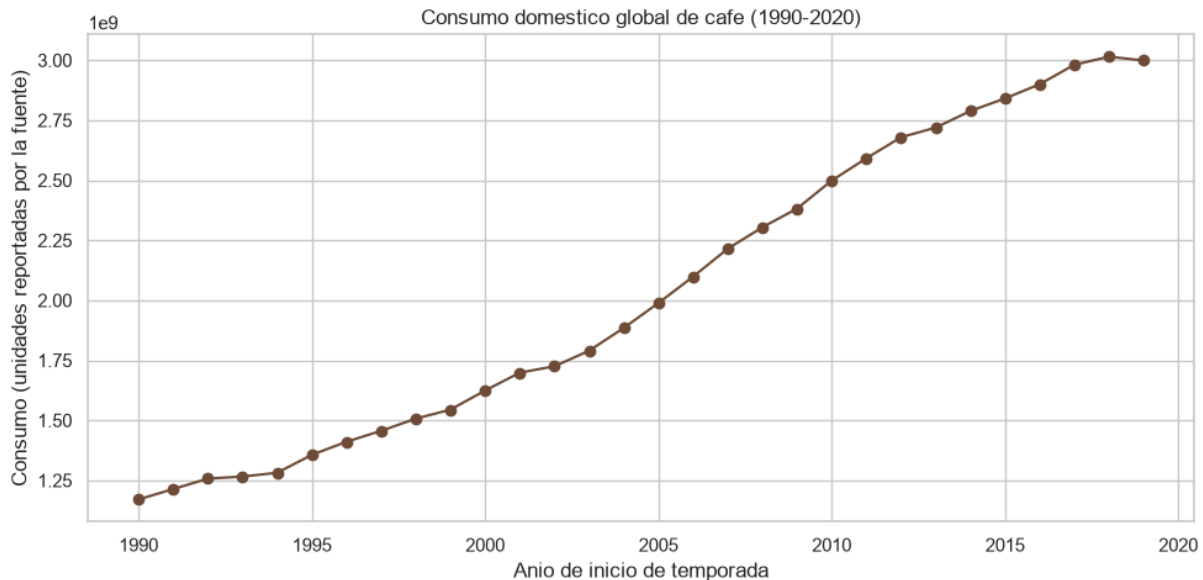
fig, ax = plt.subplots()
ax.plot(global_by_year['Year_start'], global_by_year['Consumption'],
marker='o')
ax.set_title('Consumo domestico global de cafe (1990-2020)')
ax.set_xlabel('Año de inicio de temporada')
ax.set_ylabel('Consumo (unidades reportadas por la fuente)')
fig.tight_layout()
fig.savefig(CHARTS_DIR / '01_tendencia_global.png', dpi=150)
plt.show()

growth_total = (global_by_year['Consumption'].iloc[-1] /
global_by_year['Consumption'].iloc[0] - 1)
print(f'Crecimiento acumulado del consumo global 1990/91-2019/20:
{growth_total:.1%}')

```

Figura 1

Consumo doméstico global de café (1990-2020)



Nota. El consumo global creció de forma sostenida durante las tres décadas, con una aceleración visible a partir del año 2000.

Resultado en consola:

Crecimiento acumulado del consumo global 1990/91-2019/20: 156.2%

El consumo doméstico global de café casi se triplicó entre 1990/91 y 2019/20, con un crecimiento acumulado de 156.2%. Este crecimiento no es lineal: se observa una fase de aceleración clara a partir de los años 2000, coincidiendo con el auge del consumo interno en mercados productores emergentes.

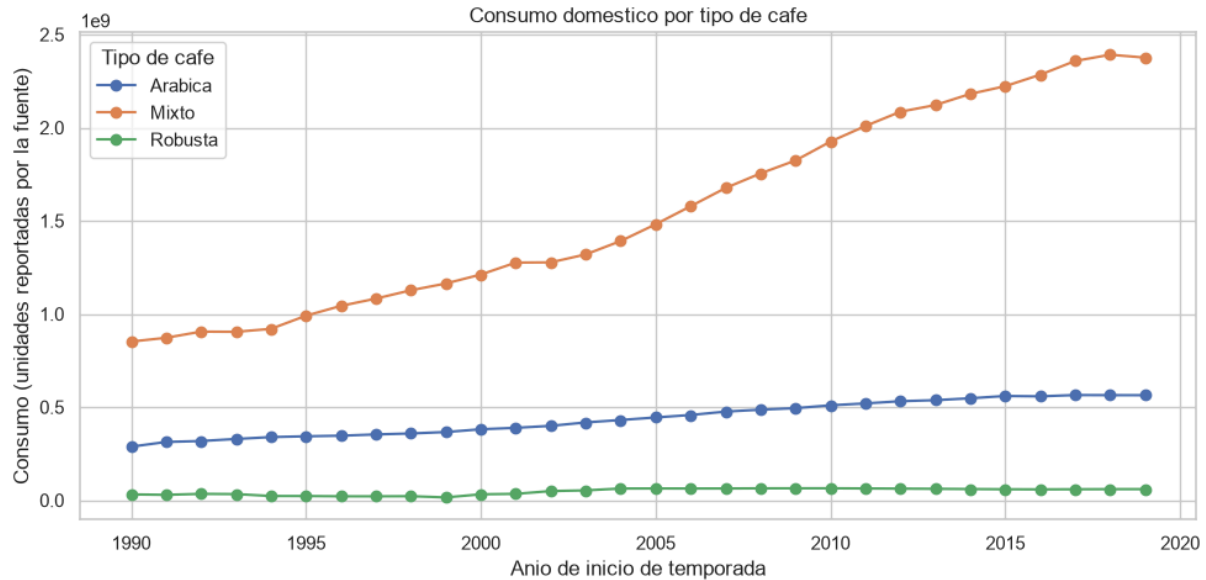
Código 5. Tendencia de consumo por tipo de café

```
by_type = df_long.groupby(['Year_start',
    'Coffee_type_grouped'])['Consumption'].sum().reset_index()

fig, ax = plt.subplots()
for t, g in by_type.groupby('Coffee_type_grouped'):
    ax.plot(g['Year_start'], g['Consumption'], marker='o', label=t)
ax.legend(title='Tipo de cafe')
ax.set_title('Consumo domestico por tipo de cafe')
ax.set_xlabel('Año de inicio de temporada')
ax.set_ylabel('Consumo (unidades reportadas por la fuente)')
fig.tight_layout()
fig.savefig(CHARTS_DIR / '02_tendencia_por_tipo.png', dpi=150)
plt.show()
```

Figura 2

Consumo doméstico por tipo de café

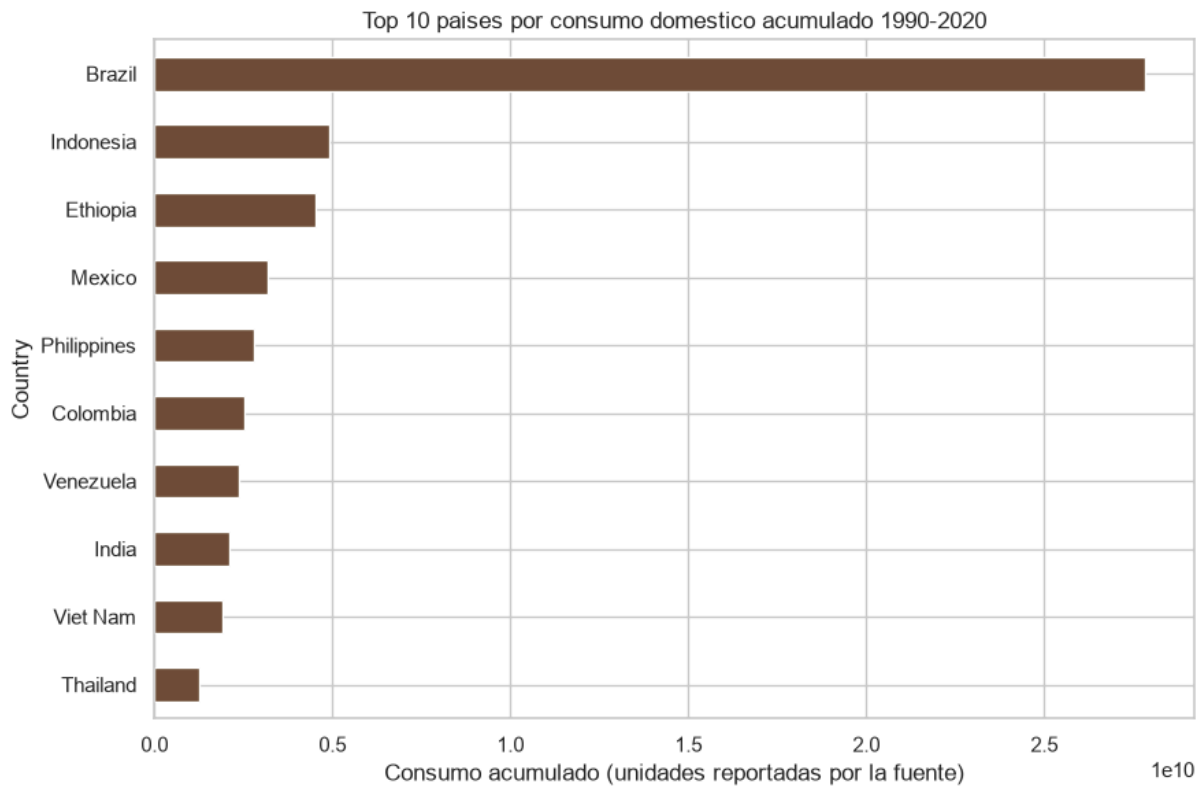


Nota. La categoría Mixto (países que reportan Arabica/Robusta o Robusta/Arabica) concentra el mayor volumen y el mayor crecimiento absoluto; Robusta se mantiene relativamente plano en todo el periodo.

Código 6. Top 10 países por consumo acumulado

```
totals =
df_wide.groupby('Country')['Total_domestic_consumption'].sum().sort_values(ascending=False)
top10 = totals.head(10)

fig, ax = plt.subplots(figsize=(9, 6))
top10.iloc[::-1].plot(kind='barh', ax=ax, color='#6f4e37')
ax.set_title('Top 10 países por consumo domestico acumulado 1990-2020')
ax.set_xlabel('Consumo acumulado (unidades reportadas por la fuente)')
fig.tight_layout()
fig.savefig(CHARTS_DIR / '03_top10_paises.png', dpi=150)
plt.show()
```

Figura 3*Top 10 países por consumo doméstico acumulado (1990-2020)***Tabla 3***Consumo doméstico acumulado por país, top 10 (1990-2020)*

País	Consumo acumulado 1990-2020
Brazil	27,824,700,000
Indonesia	4,920,480,000
Ethiopia	4,536,540,000
Mexico	3,189,660,000
Philippines	2,807,280,000
Colombia	2,536,776,384
Venezuela	2,386,067,999
India	2,093,460,000
Viet Nam	1,920,928,320
Thailand	1,248,600,000

Nota. Brasil concentra, por sí solo, un volumen de consumo doméstico acumulado muy superior al resto de los diez primeros países combinados.

3. Crecimiento y ranking de países (CAGR)

Regla de cálculo del CAGR (evita divisiones por cero): se valida que exista al menos un valor positivo en la serie del país. Si 1990/91 es 0, el CAGR se calcula desde el primer año con valor positivo registrado. Si el último año (2019/20) es 0 (el país dejó de reportar o perdió todo su consumo doméstico) o si ningún año de la serie es positivo, el CAGR se marca como NaN y el país se excluye del ranking de crecimiento, aunque se documenta aparte como caso especial en la sección de anomalías.

Código 7. Cálculo de CAGR y estado de consumo por país

```
def compute_cagr(row: pd.Series):
    series = row[SEASON_COLS].astype(float)
    positive = series[series > 0]
    if positive.empty:
        return np.nan
    start_idx = positive.index[0]
    start_val = positive.iloc[0]
    end_val = series.iloc[-1]
    n_periods = SEASON_COLS.index(series.index[-1]) -
SEASON_COLS.index(start_idx)
    if n_periods <= 0 or end_val <= 0:
        return np.nan
    return (end_val / start_val) ** (1 / n_periods) - 1

df_wide['CAGR'] = df_wide.apply(compute_cagr, axis=1)

def consumption_status(row: pd.Series):
    series = row[SEASON_COLS].astype(float)
    if (series > 0).sum() == 0:
        return 'sin_datos'
    if series.iloc[-1] <= 0:
        return 'interrumpido'
    return 'activo'

df_wide['Consumption_status'] = df_wide.apply(consumption_status, axis=1)
print(df_wide['Consumption_status'].value_counts())
print(f"Países excluidos del ranking de CAGR (CAGR = NaN):
{df_wide['CAGR'].isna().sum()}")
```

Resultado en consola:

```
Consumption_status
activo          52
sin_datos       2
interrumpido    1
Name: count, dtype: int64
Países excluidos del ranking de CAGR (CAGR = NaN): 3
```

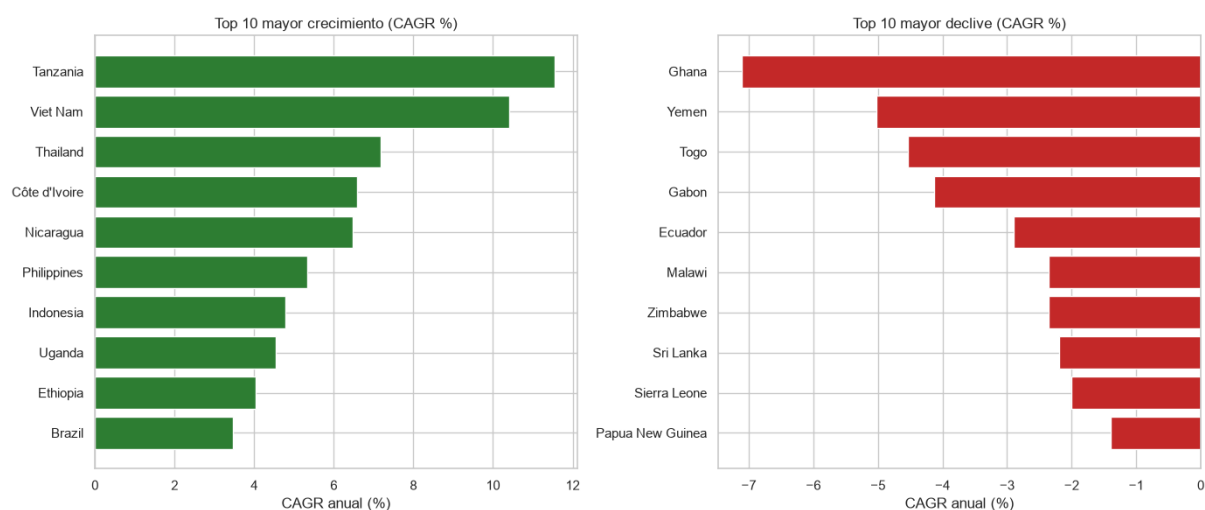
Código 8. Ranking de crecimiento anual compuesto (CAGR)

```
growth_ranking = (
    df_wide[['Country', 'Coffee_type_grouped', 'CAGR']]
    .dropna(subset=['CAGR'])
    .sort_values('CAGR', ascending=False)
)
top_gainers = growth_ranking.head(10)
top_losers = growth_ranking.tail(10).sort_values('CAGR')

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].barh(top_gainers['Country'][::-1], top_gainers['CAGR'][::-1] * 100,
color='#2e7d32')
axes[0].set_title('Top 10 mayor crecimiento (CAGR %)')
axes[0].set_xlabel('CAGR anual (%)')
axes[1].barh(top_losers['Country'][::-1], top_losers['CAGR'][::-1] * 100,
color='#c62828')
axes[1].set_title('Top 10 mayor declive (CAGR %)')
axes[1].set_xlabel('CAGR anual (%)')
fig.tight_layout()
fig.savefig(CHARTS_DIR / '04_ranking_crecimiento.png', dpi=150)
plt.show()
```

Figura 4

Top 10 países por mayor crecimiento y por mayor declive (CAGR anual, %)



Nota. Tanzania, Viet Nam y Tailandia lideran el crecimiento; Ghana, Yemen y Togo muestran el mayor declive sostenido en su consumo doméstico.

Tabla 4

Top 10 países con mayor crecimiento anual compuesto (CAGR) de consumo doméstico

País	Tipo de café	CAGR anual
Tanzania	Mixto	11.54%
Viet Nam	Mixto	10.41%
Thailand	Mixto	7.17%
Côte d'Ivoire	Robusta	6.58%

Nicaragua	Arabica	6.48%
Philippines	Mixto	5.33%
Indonesia	Mixto	4.78%
Uganda	Mixto	4.54%
Ethiopia	Arabica	4.04%
Brazil	Mixto	3.46%

4. Segmentación de países productores (clustering)

Se agrupan los 55 países según tres características de su patrón de consumo doméstico: nivel promedio (log), crecimiento (CAGR) y volatilidad (desviación estándar de las variaciones interanuales). Los 3 países sin CAGR válido (sin_datos o interrumpido) se excluyen del clustering porque no tienen una señal de crecimiento comparable, y se reportan aparte. KMeans y PCA usan random_state=42 para reproducibilidad.

Código 9. Features de segmentación y selección del número de clusters

```
def yoy_std(row: pd.Series):
    series = row[SEASON_COLS].astype(float)
    pct = series.pct_change().replace([np.inf, -np.inf], np.nan).dropna()
    return pct.std()

df_wide['Volatility'] = df_wide.apply(yoy_std, axis=1)
df_wide['Level_mean'] = df_wide[SEASON_COLS].astype(float).mean(axis=1)

cluster_input = df_wide.dropna(subset=['CAGR', 'Volatility']).copy()
excluded_from_clustering =
df_wide[~df_wide['Country'].isin(cluster_input['Country'])]
print('Países excluidos del clustering:', excluded_from_clustering)

features = cluster_input[['Level_mean', 'CAGR', 'Volatility']].copy()
features['Level_mean'] = np.log1p(features['Level_mean'])

scaler = StandardScaler()
X = scaler.fit_transform(features)

sil_scores = {}
for k in range(2, 7):
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    labels = km.fit_predict(X)
    sil_scores[k] = silhouette_score(X, labels)

best_k = max(sil_scores, key=sil_scores.get)
print('Silhouette por k:', {k: round(v, 3) for k, v in sil_scores.items()})
print('k elegido:', best_k)
```

Resultado en consola:

```
Países excluidos del clustering: ['Zambia', 'Equatorial Guinea', 'Nepal']
Silhouette por k: {2: 0.332, 3: 0.408, 4: 0.323, 5: 0.347, 6: 0.334}
k elegido: 3
```

Código 10. Segmentación con K-Means y proyección PCA

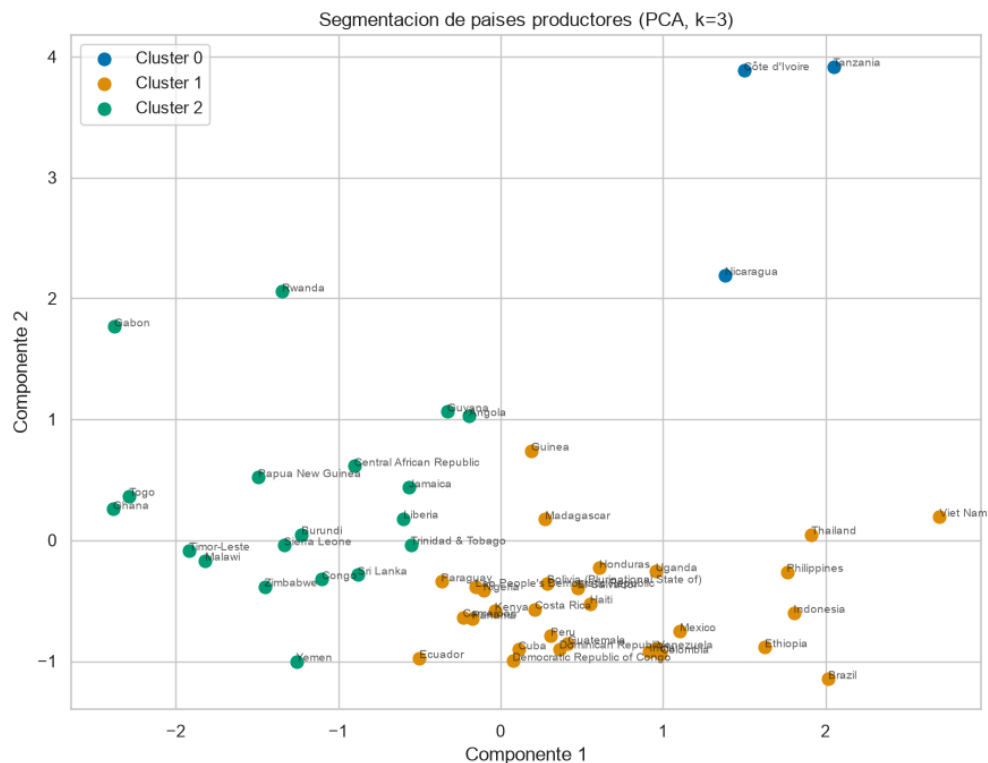
```
kmeans = KMeans(n_clusters=best_k, random_state=RANDOM_STATE, n_init=10)
cluster_input['Cluster'] = kmeans.fit_predict(X)

pca = PCA(n_components=2, random_state=RANDOM_STATE)
coords = pca.fit_transform(X)
cluster_input['PC1'] = coords[:, 0]
cluster_input['PC2'] = coords[:, 1]

fig, ax = plt.subplots(figsize=(9, 7))
palette = sns.color_palette('colorblind', n_colors=best_k)
for c in sorted(cluster_input['Cluster'].unique()):
    sub = cluster_input[cluster_input['Cluster'] == c]
    ax.scatter(sub['PC1'], sub['PC2'], label=f'Cluster {c}',
              color=palette[c])
for _, r in cluster_input.iterrows():
    ax.annotate(r['Country'], (r['PC1'], r['PC2']), fontsize=7, alpha=0.7)
ax.set_title(f'Segmentacion de paises productores (PCA, k={best_k})')
ax.set_xlabel('Componente 1')
ax.set_ylabel('Componente 2')
ax.legend()
fig.tight_layout()
fig.savefig(CHARTS_DIR / '05_clusters_pca.png', dpi=150)
plt.show()
```

Figura 5

Segmentación de países productores mediante PCA ($k = 3$)



Nota. Los ejes representan los dos primeros componentes principales de las variables estandarizadas (nivel, CAGR y volatilidad).

Código 11. Perfil descriptivo de cada cluster

```
cluster_profile = cluster_input.groupby('Cluster').agg(
    n_paises=('Country', 'count'),
    nivel_promedio=('Level_mean', 'mean'),
    cagr_promedio=('CAGR', 'mean'),
    volatilidad_promedio=('Volatility', 'mean'),
    paises=('Country', lambda s: ', '.join(sorted(s))),
).reset_index()
cluster_profile['cagr_promedio'] = (cluster_profile['cagr_promedio'] *
100).round(2)
cluster_profile['volatilidad_promedio'] =
(cluster_profile['volatilidad_promedio'] * 100).round(2)
cluster_profile
```

Tabla 5

Perfil descriptivo de los tres clusters de países productores

Cluster	N.º países	Nivel promedio	CAGR promedio	Volatilidad promedio	Países (muestra)
0	3	8,737,617	8.20%	80.85%	Côte d'Ivoire, Nicaragua, Tanzania
1	30	66,712,730	2.03%	7.92%	Bolivia (Estado Plurinacional de), Brazil, Cameroon...
2	19	627,943	-1.29%	21.66%	Angola, Burundi, Central African Republic, Congo...

Nota. El Cluster 0 agrupa países pequeños de crecimiento explosivo y alta volatilidad; el Cluster 1 concentra a la mayoría de los grandes productores, con crecimiento moderado y baja volatilidad; el Cluster 2 agrupa países de bajo nivel de consumo y tendencia decreciente. El listado completo de países por cluster se encuentra en el archivo de resultados analysis_summary.json.

5. Forecasting de consumo doméstico

Con solo 30 puntos anuales por país, ajustar 55 modelos individuales finos arriesga sobreajuste. Se compara, mediante backtesting sobre la serie global (entrenando hasta 2014/15 y evaluando 2015/16-2019/20), un modelo de tendencia lineal contra un modelo Holt (suavizado exponencial con tendencia aditiva, sin estacionalidad por tratarse de datos anuales). El método con menor error (MAPE) en el backtest se usa después para proyectar 5 años (2020/21-2024/25)

sobre: el agregado global, cada tipo de café agregado, y los 10 países de mayor consumo (no se proyectan los 55 países individualmente, para no dar una falsa sensación de precisión).

Código 12. Backtesting: tendencia lineal vs. modelo Holt

```
def linear_forecast(values: np.ndarray, periods: int):
    x = np.arange(len(values))
    coeffs = np.polyfit(x, values, 1)
    future_x = np.arange(len(values), len(values) + periods)
    return np.polyval(coeffs, future_x)

def holt_forecast(values: np.ndarray, periods: int):
    model = ExponentialSmoothing(values, trend='add', damped_trend=True)
    fit = model.fit()
    return fit.forecast(periods)

def mape(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    mask = y_true != 0
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) *
100

global_series = global_by_year.set_index('Year_start')['Consumption'].values
train, test = global_series[:-5], global_series[-5:]

lin_pred = linear_forecast(train, 5)
holt_pred = holt_forecast(train, 5)

mape_lin = mape(test, lin_pred)
mape_holt = mape(test, holt_pred)
print(f'MAPE backtest tendencia lineal: {mape_lin:.2f}%')
print(f'MAPE backtest Holt (tendencia amortiguada): {mape_holt:.2f}%')

best_method = 'linear' if mape_lin <= mape_holt else 'holt'
print('Metodo elegido para las proyecciones:', best_method)
```

Resultado en consola:

```
MAPE backtest tendencia lineal: 1.68%
MAPE backtest Holt (tendencia amortiguada): 1.48%
Método elegido para las proyecciones: holt
```

Código 13. Proyección del consumo global a 5 años

```
FUTURE_SEASONS = ['2020/21', '2021/22', '2022/23', '2023/24', '2024/25']
global_forecast = forecast_series(global_series, 5)
residual_std = np.std(test - (lin_pred if best_method == 'linear' else
holt_pred))

fig, ax = plt.subplots()
ax.plot(global_by_year['Year_start'], global_by_year['Consumption'],
marker='o', label='Historico')
future_years = list(range(2020, 2025))
ax.plot(future_years, global_forecast, marker='o', linestyle='--',
label='Forecast')
ax.fill_between(future_years, global_forecast - 1.28 * residual_std,
```

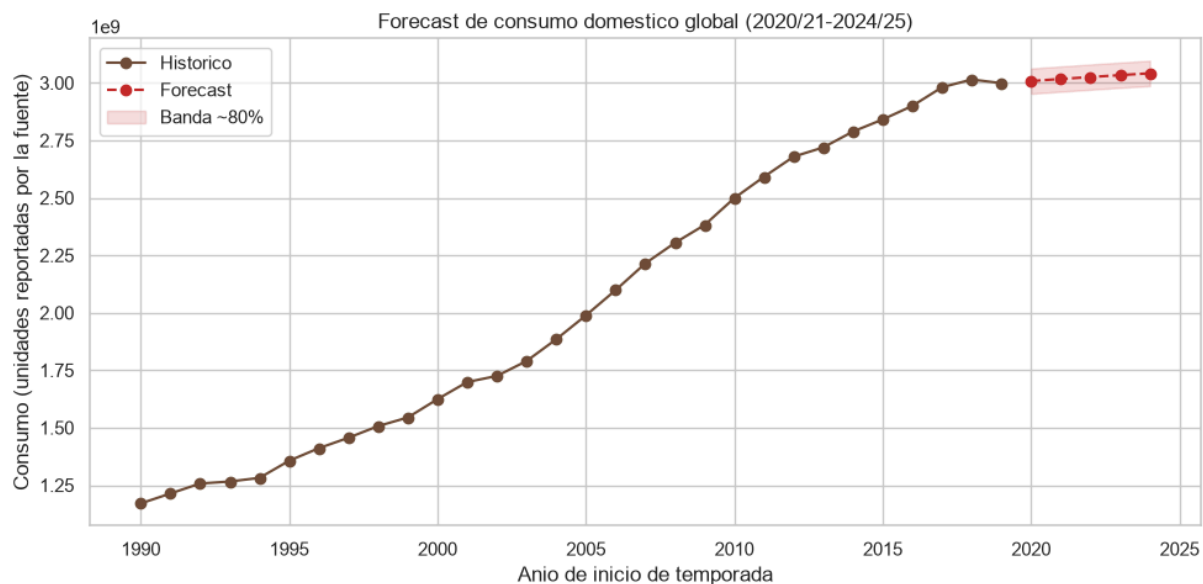
```

        global_forecast + 1.28 * residual_std, color='#c62828',
alpha=0.15,
        label='Banda ~80%')
ax.legend()
ax.set_title('Forecast de consumo domestico global (2020/21-2024/25)')
fig.tight_layout()
fig.savefig(CHARTS_DIR / '06_forecast_global.png', dpi=150)
plt.show()

```

Figura 6

Forecast de consumo doméstico global (2020/21-2024/25) con banda de incertidumbre ~80%



Código 14. Proyección por tipo de café y por país (top 10)

```

for t, g in by_type.groupby('Coffee_type_grouped'):
    series = g.set_index('Year_start')['Consumption'].reindex(range(1990,
2020))
    fc = forecast_series(series, 5)
    forecast_results[t.lower()] = {'metodo': best_method, 'seasons':
FUTURE_SEASONS,
                                'valores': [round(v, 0) for v in
fc.tolist()]}

top10_countries = totals.head(10).index.tolist()
for country in top10_countries:
    series = df_long[df_long['Country'] ==
country].set_index('Year_start')['Consumption']
    fc = forecast_series(series, 5)
    forecast_results[country.lower()] = {'metodo': best_method, 'seasons':
FUTURE_SEASONS,
                                        'valores': [round(v, 0) for v in
fc.tolist()]}

print('Entidades con forecast calculado:', list(forecast_results.keys()))

```

Resultado en consola:

```
Entidades con forecast calculado: ['global', 'arabica', 'mixto', 'robusta',  
'brazil', 'indonesia', 'ethiopia', 'mexico', 'philippines', 'colombia',  
'venezuela', 'india', 'viet nam', 'thailand']
```

6. Índice de presión de precios (SIMULADO)

El conjunto de datos no incluye precios de mercado. Para responder al pedido explícito del reto de “rangos de precios futuros” sin inventar datos de mercado como si fueran reales, se construye un índice ilustrativo con metodología transparente, diseñado como un oscilador de presión (no como un nivel de precio compuesto, que explotaría matemáticamente si se capitaliza un crecimiento sostenido durante 30 años):

$$\text{Índice}_t = 100 + k \cdot 100 \cdot (\text{crecimiento_consumo_global}_t - \text{crecimiento_promedio_histórico}) + \text{ruido}_t$$

Intuición: si el consumo doméstico global crece un año por encima de su promedio histórico, una porción mayor de la producción se queda en el país de origen ese año, ajustando el excedente exportable disponible por debajo de lo habitual, lo que se traduce en el índice como una señal de mayor presión al alza sobre precios (valor por encima de 100); si crece por debajo de su promedio histórico, la señal es de menor presión (valor por debajo de 100). k es una constante de sensibilidad ilustrativa (no calibrada con datos de mercado reales) y ruido_t es una perturbación aleatoria pequeña ($\text{random_state} = 42$) que representa la variabilidad real de precios que este único indicador no puede explicar.

Aviso explícito: este índice NO son precios reales de café, es una simulación con fines demostrativos, acotada alrededor de 100 en vez de una serie que se dispara, para que su escala se pueda leer como una señal de tensión relativa y no como un nivel de precio en dólares. Para

una versión productiva se requeriría integrar datos reales, por ejemplo el indicador compuesto de la OIC o los futuros de café de ICE.

Código 15. Construcción del índice de presión de precios simulado

```
rng = np.random.default_rng(RANDOM_STATE)
K_SENSITIVITY = 3.0
NOISE_STD_POINTS = 1.0

global_growth = global_by_year['Consumption'].pct_change().dropna().values
mean_hist_growth = global_growth.mean()

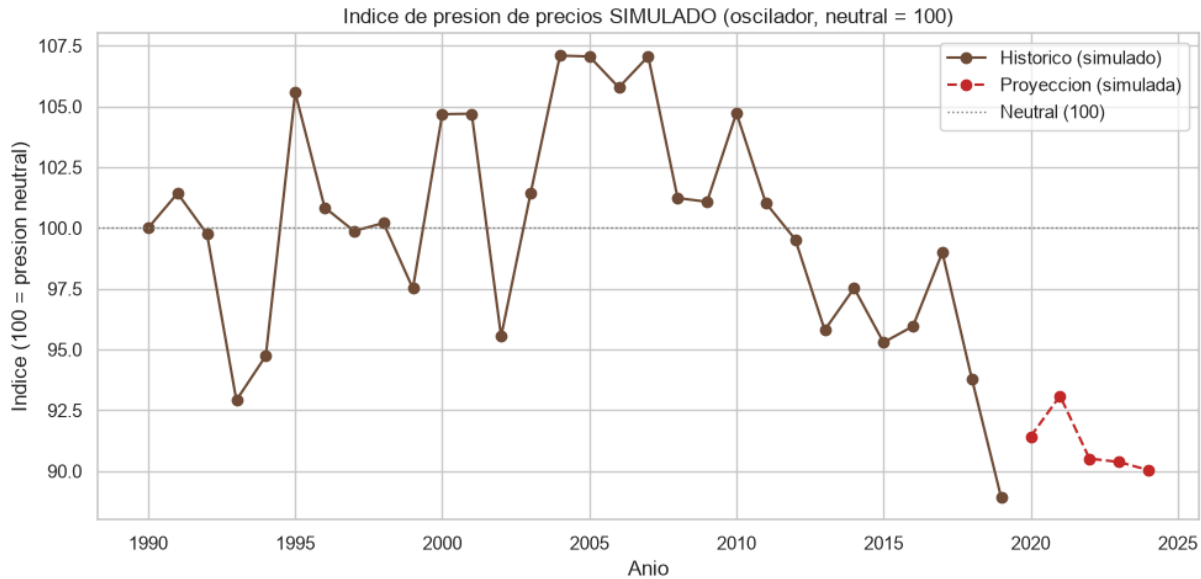
index_values = [100.0]
for g in global_growth:
    noise = rng.normal(loc=0.0, scale=NOISE_STD_POINTS)
    deviation = g - mean_hist_growth
    next_val = 100 + K_SENSITIVITY * 100 * deviation + noise
    index_values.append(next_val)

future_growth = np.diff(np.concatenate([[global_series[-1]],
global_forecast]))
index_future = []
for g in future_growth:
    noise = rng.normal(loc=0.0, scale=NOISE_STD_POINTS)
    deviation = g - mean_hist_growth
    index_future.append(100 + K_SENSITIVITY * 100 * deviation + noise)

fig, ax = plt.subplots()
ax.plot(price_index_hist['Year_start'], price_index_hist['Indice_simulado'],
marker='o', label='Historico (simulado)')
ax.plot(future_years, index_future, marker='o', linestyle='--',
label='Proyección (simulada)')
ax.axhline(100, color='gray', linestyle=':', linewidth=1, label='Neutral
(100)')
ax.legend()
ax.set_title('Indice de presion de precios SIMULADO (oscilador, neutral =
100)')
fig.tight_layout()
fig.savefig(CHARTS_DIR / '07_indice_precio_simulado.png', dpi=150)
plt.show()
```

Figura 7

Índice de presión de precios SIMULADO (oscilador, neutral = 100)



Nota. Este índice es una simulación ilustrativa y no representa precios reales de café.

Resultado en consola:

```
{'aviso': 'Indice ilustrativo simulado (oscilador de presion), NO son
precios de mercado reales.',
'neutral': '100 = presion neutral (crecimiento de consumo igual al
promedio historico 1990/91-2019/20)',
'k_sensibilidad': 3.0,
'valor_2019_20': 88.9,
'proyeccion_seasons': ['2020/21', '2021/22', '2022/23', '2023/24',
'2024/25'],
'proyeccion_valores': [91.4, 93.1, 90.5, 90.4, 90.0]}
```

7. Detección de anomalías y eventos relevantes

Código 16. Detección de anomalías interanuales (z-score > 2.5)

```
df_long['YoY_pct'] = df_long.groupby('Country')['Consumption'].pct_change()

def zscore(s: pd.Series):
    std = s.std()
    if std == 0 or np.isnan(std):
        return pd.Series(np.nan, index=s.index)
    return (s - s.mean()) / std

df_long['YoY_zscore'] =
df_long.groupby('Country')['YoY_pct'].transform(zscore)

anomalies = df_long[df_long['YoY_zscore'].abs() > 2.5].copy()
anomalies = anomalies.sort_values('YoY_zscore', key=lambda s: s.abs(),
ascending=False)
anomalies_top = anomalies[['Country', 'Season', 'Consumption', 'YoY_pct',
'YoY_zscore']]
anomalies_top
```

Tabla 6*Eventos anómalos detectados por variación interanual ($|z\text{-score}| > 2.5$)*

País	Temporada	Consumo	Variación interanual	Z-score
Democratic Republic of Congo	2019/20	11,880,000	-1.00%	-5.20
Paraguay	1993/94	1,200,000	33.33%	5.20
Liberia	1993/94	300,000	66.67%	5.20
Malawi	1999/00	60,000	-50.00%	-5.20
Côte d'Ivoire	2000/01	19,000,020	533.33%	5.20
Haiti	1991/92	19,800,000	50.00%	5.19
Tanzania	1992/93	1,152,000	448.57%	5.06
Gabon	2010/11	56,820	278.80%	4.97
Nicaragua	1991/92	8,280,000	305.88%	4.93
Nigeria	1991/92	2,145,000	30.00%	4.82
Philippines	2008/09	103,200,000	62.26%	4.76
Guinea	1992/93	3,750,000	150.00%	4.70
Zimbabwe	1996/97	240,000	-39.39%	-4.56
Venezuela	2019/20	76,500,000	-17.74%	-4.36
Burundi	2010/11	120,000	42.96%	4.10

Además de los picos puntuales, existen casos estructurales relevantes para el negocio que no siempre aparecen como pico puntual, pero que constituyen información clave:

Zambia: el consumo doméstico reportado cae a 0 de forma sostenida desde 2009/10 hasta 2019/20 (11 temporadas seguidas en cero), tras venir de un rango de 36,000-90,000 unidades. Es una interrupción de reporte o de actividad, no una fluctuación normal; se recomienda verificar con la fuente antes de usar este país en decisiones comerciales.

Equatorial Guinea y Nepal: no tienen ningún valor positivo registrado en las 30 temporadas. Es probable que sean países sin consumo doméstico material o sin cobertura de reporte de la fuente; se excluyen de los rankings de crecimiento y del clustering por no tener señal.

Estos tres casos se marcaron automáticamente con la variable Consumption_status descrita en la sección 3.

8. Conclusiones y recomendaciones de negocio

Consumo global: el consumo doméstico global creció de forma sostenida entre 1990/91 y 2019/20 (ver sección 2), lo que estructuralmente reduce el excedente exportable disponible en el tiempo si la producción no crece al mismo ritmo.

Concentración: un grupo reducido de países (Brasil, Indonesia, Etiopía, México, Filipinas, Colombia, Venezuela, India, Vietnam, Tailandia) concentra la mayor parte del consumo doméstico histórico acumulado; son los mercados donde una variación en su consumo interno tiene mayor impacto en la oferta exportable global.

Segmentación: los clusters de la sección 4 permiten priorizar de forma diferenciada. Los países del segmento de mayor crecimiento absorben cada vez más de su propia producción (señal de alerta temprana para asegurar contratos de suministro a largo plazo antes de que su excedente exportable se reduzca más). Los países del segmento en declive o estable liberan (o mantienen) más excedente exportable en el tiempo, y son candidatos a fortalecer relaciones comerciales de abastecimiento.

Forecast: la proyección a 2024/25 (sección 5) da una primera estimación cuantitativa para planear volúmenes de compra/exportación a mediano plazo, con una banda de incertidumbre explícita en vez de un número único.

Precios: el índice de presión de precios (sección 6) es una prueba de concepto que conecta la dinámica de consumo con una señal de precio; para uso real en decisiones comerciales

debe sustituirse o calibrarse con datos de mercado reales (por ejemplo, el indicador compuesto de la OIC o los futuros de café de ICE), y no usarse tal cual.

Calidad de datos: se recomienda a producto/datos verificar los tres casos señalados en la sección 7 (Zambia, Equatorial Guinea y Nepal) con la fuente original antes de tomarlos como insumo de decisiones comerciales.

9. Bonus: chatbot con IA generativa

Se desarrolló una demo funcional en `bonus_chatbot/chatbot.py`: un chatbot de línea de comandos que responde preguntas en lenguaje natural sobre este análisis (tendencias, ranking de crecimiento, clusters, forecast, índice de precios simulado), usando un patrón de tool-use con el modelo Claude sobre el resumen de datos que esta etapa exporta a `outputs/analysis_summary.json`. Si no hay una API key de Anthropic configurada, el script cae automáticamente a un modo mock/dry-run con un menú de preguntas frecuentes preconfeccionadas, para que se pueda ejecutar sin credenciales. Ver `bonus_chatbot/README.md` para instrucciones, y la sección 10 del informe (`informe_solucion.md`) para la propuesta completa de arquitectura y extensiones futuras (RAG, integración Slack/Teams, narrativas automáticas, alertas proactivas).

Código 17. Exportación del resumen de resultados para el chatbot

```
countries_summary = []
for _, row in df_wide.iterrows():
    countries_summary.append({
        'country': row['Country'],
        'coffee_type': row['Coffee_type_grouped'],
        'total_historico': int(row['Total_domestic_consumption']),
        'cagr': None if pd.isna(row['CAGR']) else round(float(row['CAGR']),
4),
        'consumption_status': row['Consumption_status'],
        'cluster': None,
    })

cluster_map = dict(zip(cluster_input['Country'],
cluster_input['Cluster'].astype(int)))
```

```

for c in countries_summary:
    if c['country'] in cluster_map:
        c['cluster'] = cluster_map[c['country']]

clusters_summary = {}
for _, r in cluster_profile.iterrows():
    clusters_summary[str(int(r['Cluster']))] = {
        'n_países': int(r['n_países']),
        'cagr_promedio_pct': float(r['cagr_promedio']),
        'volatilidad_promedio_pct': float(r['volatilidad_promedio']),
        'países': r['países'],
    }

analysis_summary = {
    'countries': countries_summary,
    'clusters': clusters_summary,
    'forecast': forecast_results,
    'price_index': price_index_summary,
    'excluidos_clustering': excluded_from_clustering,
}

with open(OUTPUTS_DIR / 'analysis_summary.json', 'w', encoding='utf-8') as f:
    json.dump(analysis_summary, f, ensure_ascii=False, indent=2)

```

El resumen consolidado (países, clusters, proyecciones e índice de precios) queda exportado en formato JSON como insumo directo del chatbot y de cualquier consumo posterior por parte de otras herramientas de negocio.